

Gemini를 위한 Blender Python 코드 생성 가이드

이 문서는 AI가 Blender CLI 환경에서 고품질 3D 모델을 생성하기 위한 Python 코드를 작성할 때 따라야 할 원칙과 패턴입니다.

1. 절대 원칙 (Critical Rules)

❌ 절대 하지 말아야 할 것

Python

```
# 잘못된 예: 헤드리스 환경에서 컨텍스트 에러 발생
bpy.ops.mesh.primitive_cube_add() # context.area가 없으면 실패
bpy.ops.object.select_all(action='SELECT') # 화면이 없으면 작동 안 함
```

✅ 반드시 해야 하는 것

Python

```
# 올바른 예: 직접 데이터 조작
import bpy
import bmesh

# 새 메쉬 생성
mesh = bpy.data.meshes.new("MyMesh")
obj = bpy.data.objects.new("MyObject", mesh)
bpy.context.collection.objects.link(obj)

# BMesh로 정점 조작
bm = bmesh.new()
bm.from_mesh(mesh)
# ... 작업 수행
bm.to_mesh(mesh)
bm.free()
```

2. 휴머노이드 생성 템플릿 (핵심)

A. 베이스 메쉬 로드 방식 (권장)

Python

```
import bpy
import os

# 미리 제작된 베이스 메쉬 불러오기
base_path = "/path/to/assets/base_meshes/humanoid.fbx"
if os.path.exists(base_path):
    bpy.ops.import_scene.fbx(filepath=base_path)
    base_obj = bpy.context.selected_objects[0]
    base_obj.name = "Base_Humanoid"
else:
    print("ERROR: Base mesh not found!")
```

B. 프로시저럴 생성 방식 (고급)

Python

```
import bpy
import bmesh
from mathutils import Vector

def create_humanoid_base():
    """A-포즈 휴머노이드 베이스 생성"""
    mesh = bpy.data.meshes.new("Humanoid")
    obj = bpy.data.objects.new("Humanoid", mesh)
    bpy.context.collection.objects.link(obj)

    bm = bmesh.new()

    # 몸통 (8개 정점의 박스)
    torso_verts = [
        bm.verts.new(Vector((-0.3, -0.2, 0.0))), # 하단 좌후
        bm.verts.new(Vector((0.3, -0.2, 0.0))), # 하단 우후
        bm.verts.new(Vector((0.3, 0.2, 0.0))), # 하단 우전
        bm.verts.new(Vector((-0.3, 0.2, 0.0))), # 하단 좌전
        bm.verts.new(Vector((-0.25, -0.15, 1.2))), # 상단 좌후
        bm.verts.new(Vector((0.25, -0.15, 1.2))), # 상단 우후
        bm.verts.new(Vector((0.25, 0.15, 1.2))), # 상단 우전
        bm.verts.new(Vector((-0.25, 0.15, 1.2))), # 상단 좌전
    ]

    # 면 생성 (하단, 상단, 측면들)
    bm.faces.new([torso_verts[0], torso_verts[1], torso_verts[2], torso_verts[3]])
    bm.faces.new([torso_verts[4], torso_verts[7], torso_verts[6], torso_verts[5]])
    # ... (측면 4개 추가)
```

```
bm.to_mesh(mesh)
bm.free()
return obj
```

```
humanoid = create_humanoid_base()
```

3. 원자적 조작 패턴

A. Extrude (면 뽑아내기)

Python

```
import bmesh

def extrude_face_by_normal(obj, face_index, distance):
    """특정 면을 노멀 방향으로 돌출"""
    mesh = obj.data
    bm = bmesh.new()
    bm.from_mesh(mesh)
    bm.faces.ensure_lookup_table()

    face = bm.faces[face_index]
    normal = face.normal

    # Extrude
    ret = bmesh.ops.extrude_face_region(bm, geom=[face])
    extruded_verts = [v for v in ret['geom'] if isinstance(v, bmesh.types.BMVert)]

    # 이동
    for v in extruded_verts:
        v.co += normal * distance

    bm.to_mesh(mesh)
    bm.free()

# 사용 예: 상단 면(인덱스 5)을 0.3만큼 뽑기
extrude_face_by_normal(humanoid, 5, 0.3)
```

B. Inset (면 안쪽에 새 면 생성)

Python

```
def inset_face(obj, face_index, thickness=0.1):
    """면을 안쪽으로 넣기"""
```

```

mesh = obj.data
bm = bmesh.new()
bm.from_mesh(mesh)
bm.faces.ensure_lookup_table()

face = bm.faces[face_index]
bmesh.ops.inset_individual(bm, faces=[face], thickness=thickness)

bm.to_mesh(mesh)
bm.free()

```

C. 시맨틱 선택 (Semantic Selection)

Python

```

def select_top_face(obj):
    """가장 위쪽(Z축 최대)을 향하는 면 찾기"""
    mesh = obj.data
    bm = bmesh.new()
    bm.from_mesh(mesh)

    top_face = None
    max_z = -float('inf')

    for face in bm.faces:
        center_z = sum(v.co.z for v in face.verts) / len(face.verts)
        if center_z > max_z and face.normal.z > 0.5: # 위를 향하는 면
            max_z = center_z
            top_face = face

    bm.free()
    return bm.faces.index(top_face) if top_face else None

```

4. 재질 및 색상

A. PBR 재질 설정

Python

```

def set_metallic_material(obj, base_color=(0.2, 0.4, 0.8, 1.0), metallic=0.9, r
    """금속성 재질 적용"""
    mat = bpy.data.materials.new(name="MetallicMat")
    mat.use_nodes = True
    nodes = mat.node_tree.nodes

```

```

bsdf = nodes.get("Principled BSDF")
bsdf.inputs['Base Color'].default_value = base_color
bsdf.inputs['Metallic'].default_value = metallic
bsdf.inputs['Roughness'].default_value = roughness

if obj.data.materials:
    obj.data.materials[0] = mat
else:
    obj.data.materials.append(mat)

set_metallic_material(humanoid, base_color=(0.1, 0.3, 0.9, 1.0))

```

B. 버텍스 컬러 (로우폴리 스타일)

Python

```

def apply_vertex_gradient(obj, top_color=(1, 1, 1), bottom_color=(0.2, 0.2, 0.2)
    """상단은 밝고 하단은 어둡게"""
    mesh = obj.data
    if not mesh.vertex_colors:
        mesh.vertex_colors.new()

    color_layer = mesh.vertex_colors.active

    # Z축 범위 계산
    min_z = min(v.co.z for v in mesh.vertices)
    max_z = max(v.co.z for v in mesh.vertices)
    z_range = max_z - min_z

    for poly in mesh.polygons:
        for loop_idx in poly.loop_indices:
            loop = mesh.loops[loop_idx]
            vert = mesh.vertices[loop.vertex_index]

            # 높이에 따른 보간
            t = (vert.co.z - min_z) / z_range if z_range > 0 else 0
            color = tuple(bottom_color[i] * (1-t) + top_color[i] * t for i in range(3))
            color_layer.data[loop_idx].color = (*color, 1.0)

```

5. 최종 최적화 파이프라인

Python

```

def finalize_model(obj):
    """게임 엔진용 최적화"""
    # 1. 모든 모디파이어 적용
    bpy.context.view_layer.objects.active = obj
    for mod in obj.modifiers:
        bpy.ops.object.modifier_apply(modifier=mod.name)

    # 2. 중복 정점 제거
    bpy.ops.object.mode_set(mode='EDIT')
    bpy.ops.mesh.select_all(action='SELECT')
    bpy.ops.mesh.remove_doubles(threshold=0.001)
    bpy.ops.object.mode_set(mode='OBJECT')

    # 3. 노멀 재계산
    bpy.ops.object.shade_smooth()
    obj.data.use_auto_smooth = True
    obj.data.auto_smooth_angle = 0.523599 # 30도

finalize_model(humanoid)

```

6. 스크린샷 및 검증

Python

```

def capture_screenshot(output_path="/tmp/model_view.png"):
    """현재 뷰 렌더링"""
    scene = bpy.context.scene
    scene.render.image_settings.file_format = 'PNG'
    scene.render.filepath = output_path

    # 카메라 자동 배치
    bpy.ops.object.camera_add(location=(3, -3, 2))
    camera = bpy.context.object
    camera.rotation_euler = (1.1, 0, 0.785)
    scene.camera = camera

    # 렌더링
    bpy.ops.render.render(write_still=True)
    return output_path

screenshot = capture_screenshot()
print(f"Screenshot saved: {screenshot}")

```

7. 에러 처리 및 로깅

Python

```
import traceback

def safe_execute(func, *args, **kwargs):
    """안전한 함수 실행"""
    try:
        result = func(*args, **kwargs)
        print(f"✓ {func.__name__} succeeded")
        return result
    except Exception as e:
        print(f"✗ {func.__name__} failed: {str(e)}")
        traceback.print_exc()
        return None

# 사용 예
safe_execute(create_humanoid_base)
```

8. Gemini를 위한 코드 생성 체크리스트

생성한 코드가 다음을 만족하는지 확인하세요:

- ☐ bpy.ops 를 최소화하고 bpy.data 와 bmesh 를 우선 사용했는가?
- ☐ 모든 새 오브젝트를 bpy.context.collection.objects.link() 로 씬에 추가했는가?
- ☐ BMesh 작업 후 반드시 bm.to_mesh() 와 bm.free() 를 호출했는가?
- ☐ 에러 처리(try-except)를 포함했는가?
- ☐ 생성된 모델의 통계(정점 수, 면 수)를 출력하여 검증 가능하게 했는가?